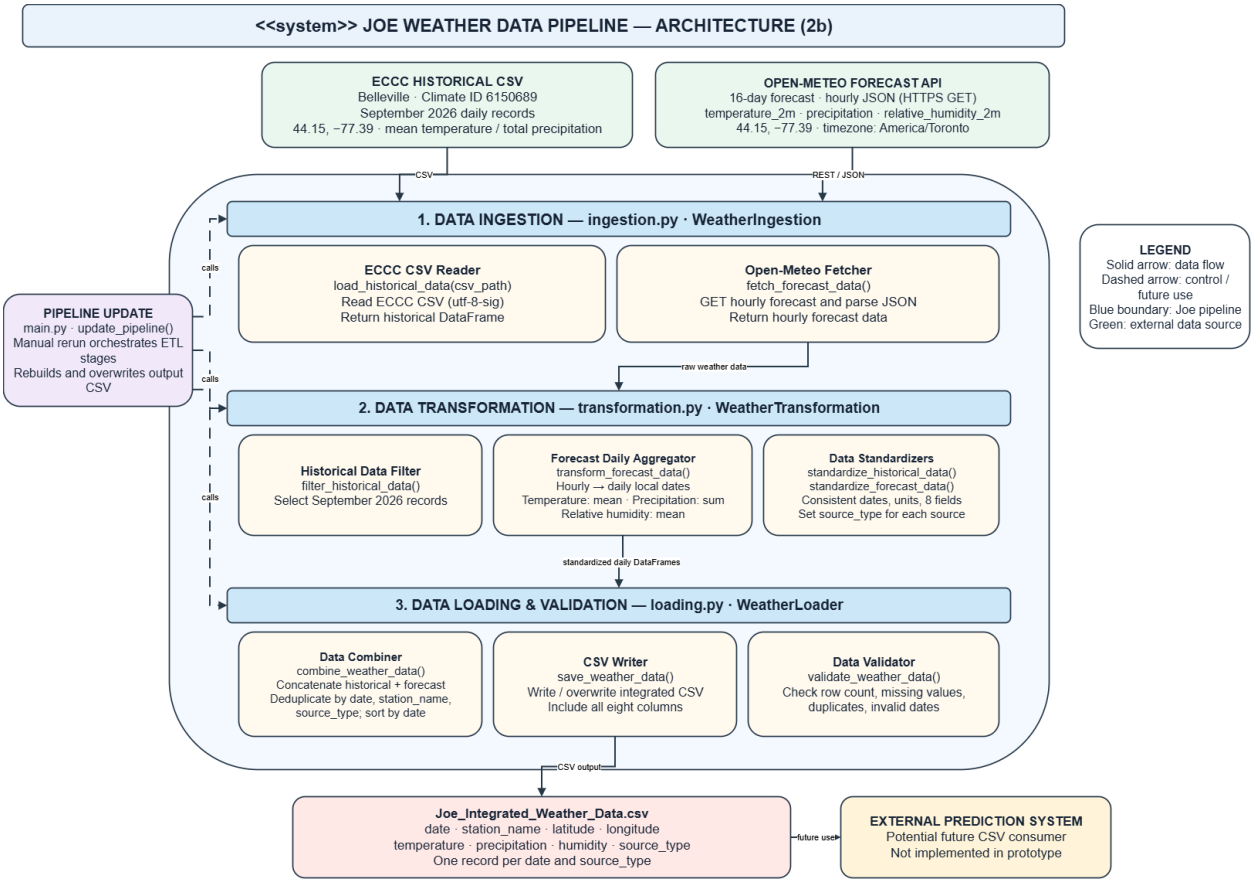
2(a). Flat-Table Data Model

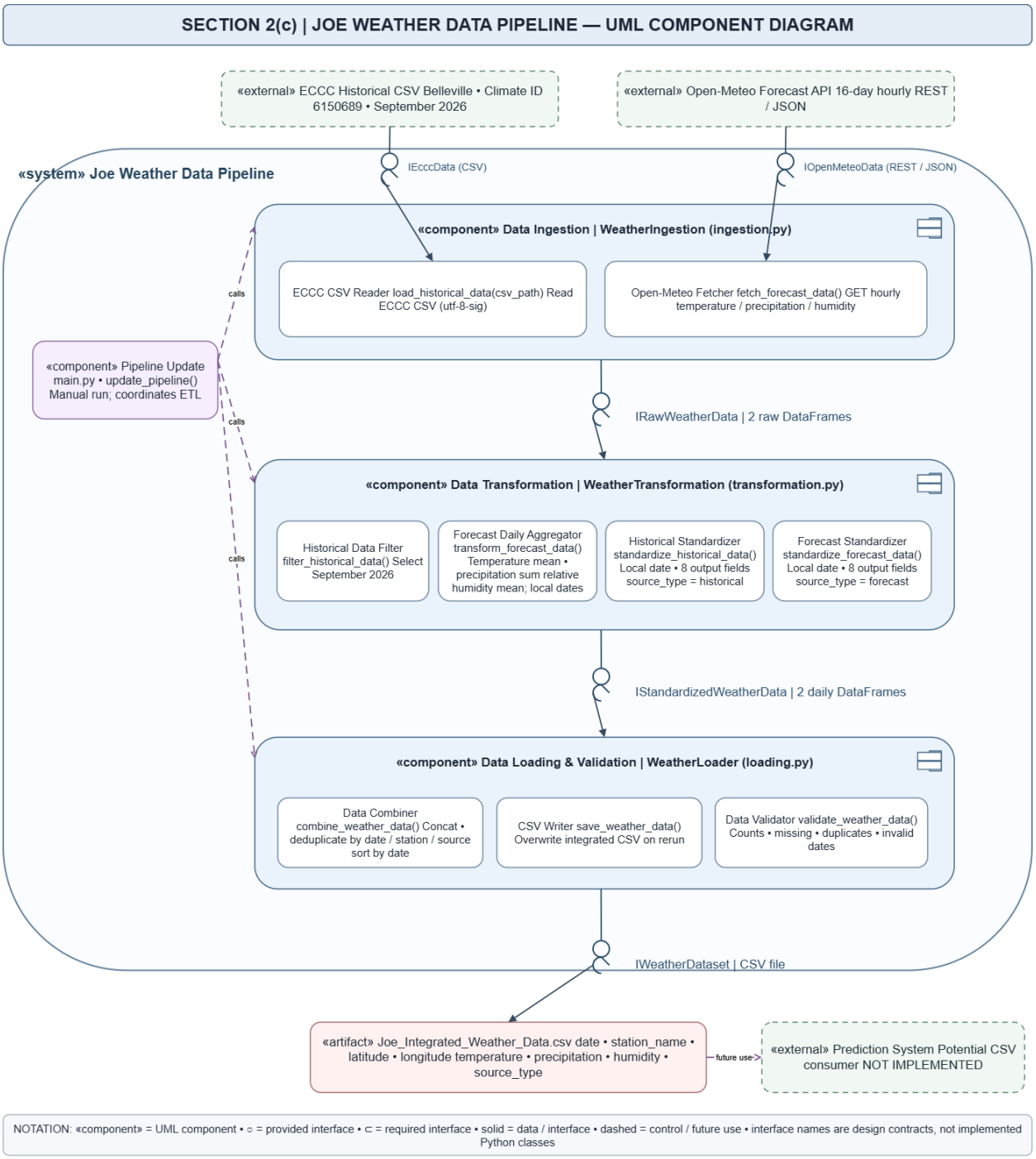
Field name	Data type	Description
date	Date	Date of the historical observation or forecast, in YYYY-MM-DD format
station_name	String	Name of the weather station
latitude	Float	Station latitude in decimal degrees
longitude	Float	Station longitude in decimal degrees
temperature	Float	Daily mean temperature in °C
precipitation	Float	Total daily precipitation in mm
humidity	Float, nullable	Daily average relative humidity in %
source_type	String	Identifies the record as historical or forecast

I designed a flat-table data model to integrate historical weather observations from Environment and Climate Change Canada (ECCC) with forecast data from Open-Meteo. Each row represents one daily weather record for the Belleville station. The source_type field distinguishes historical observations from forecasts. Missing values are preserved to avoid introducing inaccurate weather measurements.

2(b). Architectural Diagram



2(c): UML Component Diagram



2(d). List of Interfaces and Their Types

ID	Interface	Type	Between	Contract

I1	**IEcccData**	**File (CSV, UTF-8-SIG)**	**ECCC Historical CSV → Data Ingestion**	**Reads Belleville daily weather observations using load_historical_data(). Missing measurements are preserved.**
I2	**IOpenMeteoData**	**Web Service (REST, HTTPS GET, JSON)**	**Open-Meteo API → Data Ingestion**	**Requests 16 days of hourly temperature, precipitation, and relative humidity using Belleville coordinates and timezone.**
I3	**IRawWeatherData**	**Internal Data (Pandas DataFrame + JSON)**	**Data Ingestion → Data Transformation**	**Transfers historical observations and the raw forecast response for filtering and transformation.**
I4	**IStandardizedWeatherData**	**Internal Data (Pandas DataFrames)**	**Data Transformation → Data Loading**	**Transfers two standardized daily DataFrames with the same eight-column structure and consistent dates.**
I5	**IWeatherDataset**	**Output File (CSV)**	**Data Loading → Integrated CSV**	**Produces Joe_Integrated_Weather_Data.csv containing the combined weather records.**
I6	**IHistoricalDailyData**	**Internal Data (Pandas DataFrame)**	**Historical Data Filter → Historical Data Standardizer**	**Transfers September 2026 historical records for column renaming, date conversion, and source identification.**
I7	**IDailyForecastData**	**Internal Data (Pandas DataFrame)**	**Forecast Daily Aggregator → Forecast**	**Transfers daily forecast records containing mean temperature, total precipitation, and mean relative humidity.**

			Data Standardizer	
I8	**ICombinedWeatherData**	**Internal Data (Pandas DataFrame)**	**Data Combiner → CSV Writer and Data Validator**	**Transfers the combined, deduplicated, date-sorted dataset for CSV output and validation.**

2(e). Component Roles and Main Functions

1. Data Ingestion Component (ingestion.py)

The Data Ingestion component collects weather data from two sources: Environment and Climate Change Canada (ECCC) and Open-Meteo. The WeatherIngestion class contains two main methods:

- load_historical_data(csv_path): Reads the ECCC historical weather CSV using Pandas and returns a DataFrame containing the historical observations.
- fetch_forecast_data(): Sends an HTTP GET request to Open-Meteo using the Requests library. It retrieves 16 days of hourly temperature, precipitation, and relative humidity forecasts in JSON format.

2. Data Transformation Component (transformation.py)

The Data Transformation component processes both datasets and converts them into a consistent daily format. The WeatherTransformation class contains four main methods:

- filter_historical_data(data): Filters the historical ECCC dataset to retain records from September 2026.
- transform_forecast_data(forecast_data): Converts the hourly forecast into daily records by calculating mean temperature, total precipitation, and mean relative humidity for each date.
- standardize_historical_data(september_data): Renames the historical columns to match the final data model, converts the date field, and adds the source_type value historical. Missing humidity values are preserved.
- standardize_forecast_data(daily_forecast): Converts the daily forecast into the same eight-column structure, adds the Belleville station information, and assigns source_type as forecast.

How the dates are handled: ECCC provides daily observations, while Open-Meteo provides hourly forecasts. The forecast timestamps are converted into local calendar dates and grouped by date to produce one daily forecast record. Both datasets then use the same date format. Historical and forecast records are combined vertically rather than joined side by side. The source_type field distinguishes their origins, and duplicates are identified using date, station_name, and source_type. This allows the pipeline to retain records from both sources without incorrectly matching unrelated dates.

3. Data Loading and Validation Component (loading.py)

The Data Loading component combines the standardized datasets, saves the results, and checks data quality. The WeatherLoader class contains three main methods:

- combine_weather_data(historical, forecast): Concatenates both DataFrames, removes duplicate records based on date, station name, and source type, and sorts the records chronologically.
- save_weather_data(combined_data, output_path): Saves the integrated dataset as Joe_Integrated_Weather_Data.csv. Running the pipeline again overwrites the existing output with refreshed data.
- validate_weather_data(combined_data): Reports the total number of records, duplicate records, missing values, and invalid dates.

4. Pipeline Update Component (main.py)

The update_pipeline() function coordinates the entire ETL process. It calls the ingestion, transformation, and loading methods in the correct order. The pipeline runs when main.py is executed manually, allowing the latest available forecast to be retrieved and the integrated CSV to be regenerated. Automatic scheduling has not been implemented.

5. External Prediction System

The External Prediction System represents a potential future application that could read the integrated CSV and use the prepared weather data for prediction. It is included to show how the pipeline could support another system, but the prediction system itself is outside the scope of the implemented prototype.

2(f). Design Decisions and Trade-offs

1. Using a Flat-Table Data Model

I chose a flat-table structure because it allows historical observations and forecast data to be stored in one consistent format. Each record contains eight fields: date, station name, latitude, longitude, temperature, precipitation, humidity, and source type.

Trade-off: A flat table is easy to understand, analyze, and export, but it may repeat station information across multiple rows. A relational database would reduce repetition but introduce unnecessary complexity for this prototype.

2. Separating the Pipeline into ETL Components

I divided the system into Data Ingestion, Data Transformation, and Data Loading. Each component has a separate Python file and a specific responsibility. This makes the pipeline easier to test, maintain, and modify.

Trade-off: Using multiple modules requires more coordination than writing everything in one script, but it improves organization and allows individual components to be updated without rewriting the entire program.

3. Using Pandas for Data Processing

I used Pandas to filter historical records, aggregate hourly forecasts into daily values, standardize columns, combine datasets, and remove duplicates.

Trade-off: Pandas makes data processing straightforward, but it loads data into memory. This is suitable for the small prototype but could become inefficient with very large datasets.

4. Standardizing Dates and Weather Measurements

ECCC provides daily historical observations, while Open-Meteo provides hourly forecasts. I converted the forecast timestamps into daily dates and calculated mean temperature, total precipitation, and mean relative humidity. Both sources are then represented using consistent field names and date formats.

Trade-off: Daily aggregation makes the datasets compatible, but it removes hourly weather details. Also, historical and forecast measurements may differ in how they are collected or calculated.

5. Combining Historical and Forecast Records

I used vertical concatenation to combine the two standardized datasets. The source_type field identifies whether each record is historical or forecast. Duplicate records are checked using the date, station name, and source type.

Trade-off: This approach preserves records from both sources and avoids incorrectly joining unrelated dates. However, it does not automatically resolve conflicts between historical and forecast values if their date ranges overlap.

6. Using CSV Instead of a Database

I selected CSV as the output format because it is simple, portable, and supported by Pandas and many data-analysis applications.

Trade-off: CSV files are easy to create and share, but they do not provide database features such as concurrent access, advanced queries, or transaction management.

7. Manual Pipeline Updating

I implemented update_pipeline() in main.py to execute the ETL stages in sequence. Running the program again retrieves the latest available forecast and regenerates the integrated CSV.

Trade-off: Manual execution is simple and sufficient for demonstrating the prototype, but it requires someone to rerun the program. A future improvement would be to automate execution using a scheduler.

8. Preserving Missing Values

I preserved missing weather measurements rather than replacing them with zero. For example, historical humidity values that are unavailable remain missing in the final dataset.

Trade-off: This avoids introducing false weather measurements, but any future prediction system must handle missing values before using the dataset.

I can see the rest of your friend's Design Patterns table now. Your friend included five patterns, while we currently have three.

Your professor requires a minimum of three, so yours already meets that requirement. But we can make your section more detailed by using four patterns that actually fit your Python implementation.

2(g). Design Patterns Used

Pattern	Category	Where It Is Used in My Project	Why It Was Chosen
1. Multi-Layer Pattern	Architectural	The pipeline separates responsibilities into Data Ingestion,	Changes to one processing stage can be made with minimal

(Separation of Concerns)		Data Transformation, and Data Loading, coordinated by main.py.	changes to other stages. This improves maintainability and organization.
2. Pipes and Filters	Architectural	Data flows sequentially through Ingestion → Transformation → Loading. Each stage performs specific processing operations.	Allows each processing stage to be understood, tested, and modified independently.
3. Adapter	Structural	The standardize_historical_data() and standardize_forecast_data() methods convert two different source formats into a common eight-column structure.	Makes ECCC historical data and Open-Meteo forecast data compatible without changing their original sources.
4. Facade	Structural	The update_pipeline() function in main.py coordinates the ETL components through one entry point.	Simplifies execution and hides the sequence of individual processing operations from the person running the program.

3. Prototype Implementation and Execution

3.1. Prototype Overview

I developed a Python-based ETL (Extract, Transform, Load) prototype for the Joe Weather Data Pipeline. The system integrates historical weather observations from Environment and Climate Change Canada (ECCC) with forecast data from the Open-Meteo API for Belleville, Ontario.

The prototype consists of three main Python modules: ingestion.py, transformation.py, and loading.py. These modules are coordinated by main.py, which executes the complete pipeline.

The purpose of the prototype is to demonstrate how weather data from different sources can be collected, processed, standardized, validated, and stored in one integrated CSV

dataset. The resulting dataset is designed to support future weather prediction applications.

3.2. Tools and Technologies

The following tools and technologies were used to develop and execute the prototype.

Tool / Technology	Purpose
Python	Main programming language used to develop and execute the ETL pipeline.
Pandas	Used for filtering historical records, aggregating forecast data, standardizing columns, combining datasets, and validating results.
Requests	Python library used to retrieve forecast information from the Open-Meteo API through HTTP requests.
ECCC Historical Weather CSV	Provides historical daily weather observations for Belleville, Ontario.
Open-Meteo REST API	Provides hourly forecast measurements for temperature, precipitation, and relative humidity.
CSV	Used to store the final integrated weather dataset in a portable format.
Visual Studio Code	Development environment used to write, organize, test, and execute the Python modules.

3.3. Prototype Files and Structure

The prototype was organized into separate Python files to support modularity, maintainability, and separation of responsibilities.

File	Description
main.py	Contains the update_pipeline() function, which coordinates the complete ETL workflow.
ingestion.py	Contains the WeatherIngestion class, responsible for reading the ECCC CSV and retrieving Open-Meteo forecast data.

transformation.py	Contains the WeatherTransformation class, responsible for filtering historical records, aggregating hourly forecasts, and standardizing both datasets.
loading.py	Contains the WeatherLoader class, responsible for combining weather records, validating the dataset, and saving the output.
ECCC historical CSV	Input file containing the original daily historical weather observations.
Joe_Integrated_Weather_Data.csv	Output file containing the integrated and standardized weather dataset.

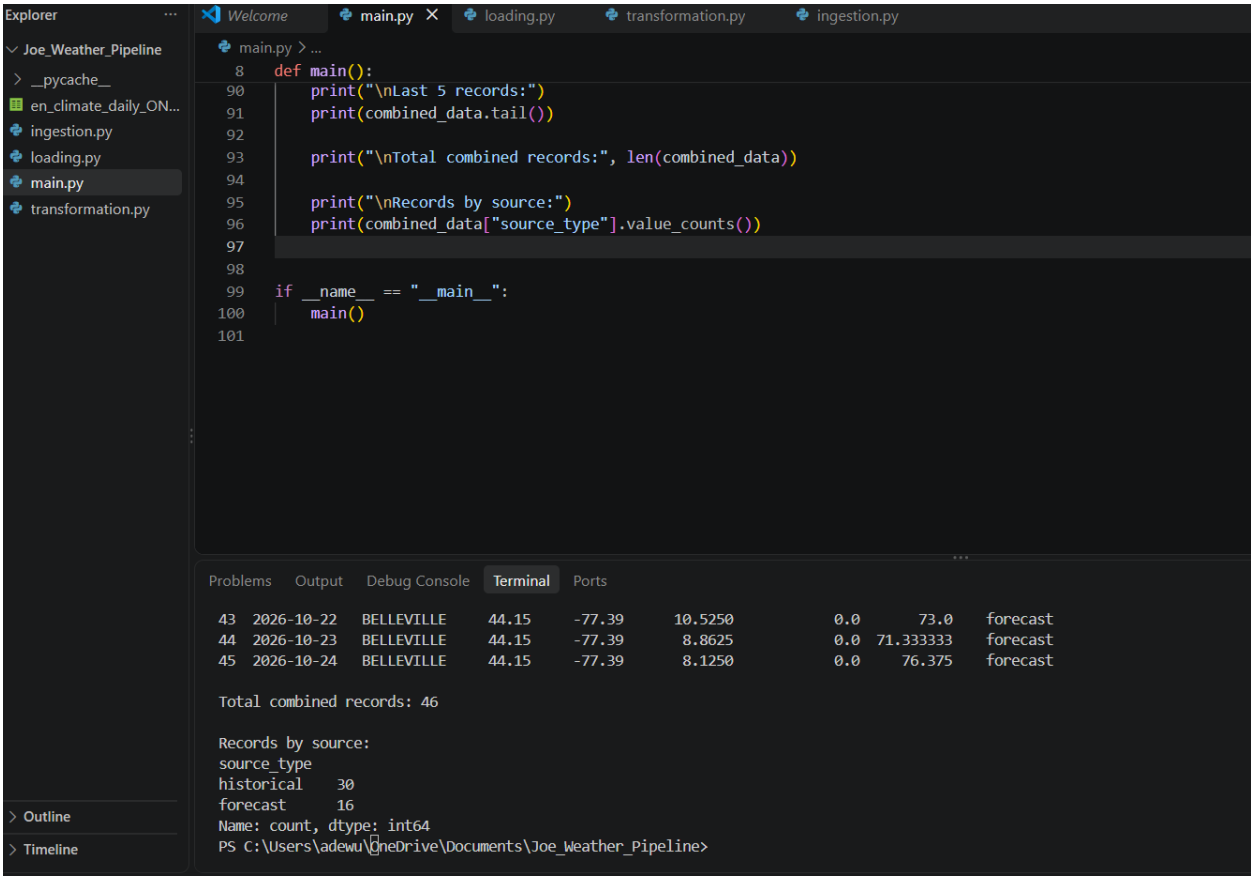


Figure 3.1: Python prototype project structure.

3.4. Running the Prototype

The prototype can be executed from the project directory using the following command:

python main.py

The update_pipeline() function coordinates the following operations:

1. **Historical Data Ingestion:** Reads the ECCC historical weather CSV for Belleville, Ontario, using Pandas.
2. **Forecast Data Ingestion:** Sends an HTTP request to the Open-Meteo API to retrieve 16 days of hourly temperature, precipitation, and relative humidity forecasts.
3. **Historical Data Filtering:** Selects historical observations recorded during September 2026.
4. **Forecast Data Aggregation:** Converts hourly forecast timestamps into daily dates and calculates average temperature, total precipitation, and average relative humidity.
5. **Data Standardization:** Converts both datasets into a consistent eight-column structure with standardized field names and dates.
6. **Data Integration:** Combines the standardized historical and forecast records into one dataset using vertical concatenation.
7. **Data Validation:** Checks for duplicate records, invalid dates, and missing measurements.
8. **Data Export:** Saves the integrated dataset as Joe_Integrated_Weather_Data.csv.

The pipeline can be executed again to retrieve updated forecast information and regenerate the integrated CSV file. The current prototype uses manual execution rather than an automated scheduler.

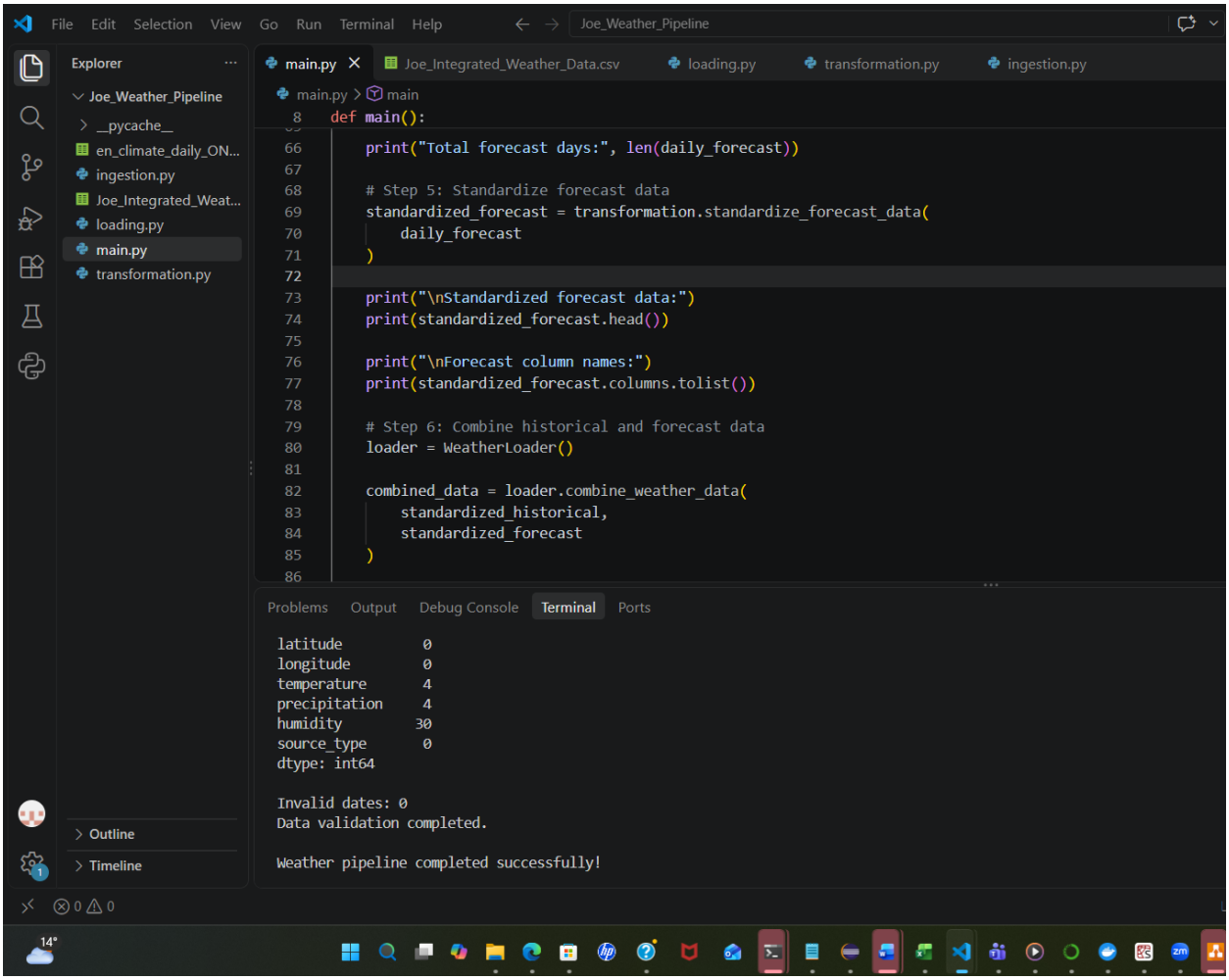


Figure 3.2: Successful execution of the Joe Weather Data Pipeline.

3.5. Prototype Results and Validation

During a successful prototype test, the system generated 46 weather records, consisting of 30 historical observations from September 2026 and 16 daily forecast records from Open-Meteo.

The following table summarizes the validation results obtained during that execution.

Validation Metric	Result
Historical weather records	30
Forecast weather records	16
Total integrated records	46

Duplicate records	0
Invalid dates	0
Missing historical temperature values	4
Missing historical precipitation values	4
Missing historical humidity values	30

The validation results indicate that no duplicate records or invalid dates were detected during the test. However, some historical weather measurements were unavailable.

Rather than replacing missing values with zero, the prototype preserves them as missing data to avoid introducing inaccurate weather measurements. Historical humidity values are also left missing because the historical dataset does not provide the corresponding measurements in the standardized output.

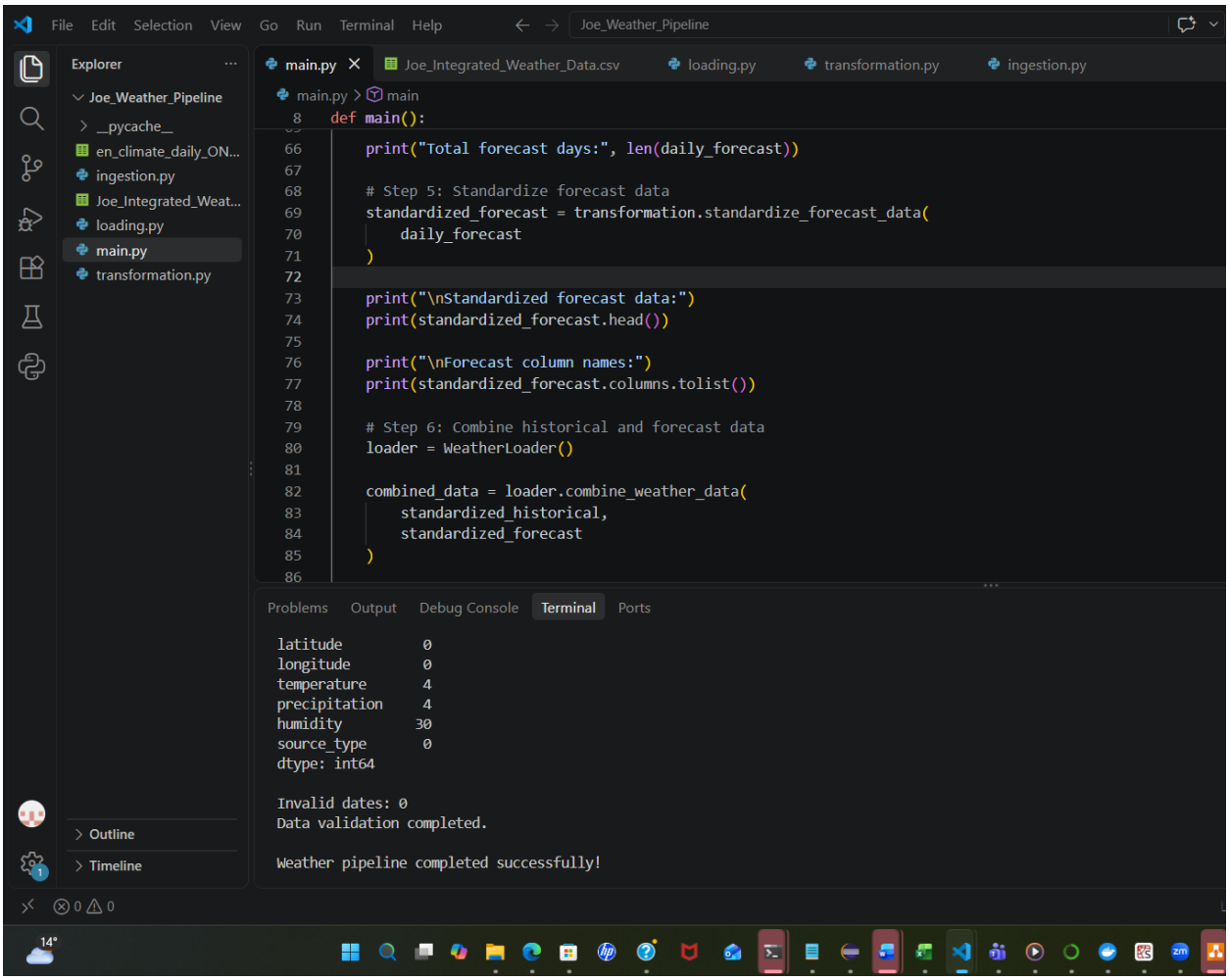


Figure 3.3: Weather dataset validation results.

3.6. Integrated Weather Dataset

The final output is stored in Joe_Integrated_Weather_Data.csv, which contains the following eight standardized columns:

Column	Description
date	Date of the weather observation or forecast.
station_name	Name of the weather station or location.
latitude	Latitude of the weather location.
longitude	Longitude of the weather location.
temperature	Daily temperature value in degrees Celsius.
precipitation	Daily precipitation amount in millimetres.
humidity	Daily relative humidity percentage, where available.
source_type	Identifies whether the record comes from historical observations or forecast data.

The historical and forecast records are combined vertically because they represent observations and predictions for different dates. The source_type column distinguishes the two categories, while standardized dates allow the dataset to be organized chronologically.

The integrated dataset can be opened in spreadsheet applications or processed by other Python programs. It provides a structured data source for a future weather prediction system, although the prediction system itself is outside the scope of this prototype.

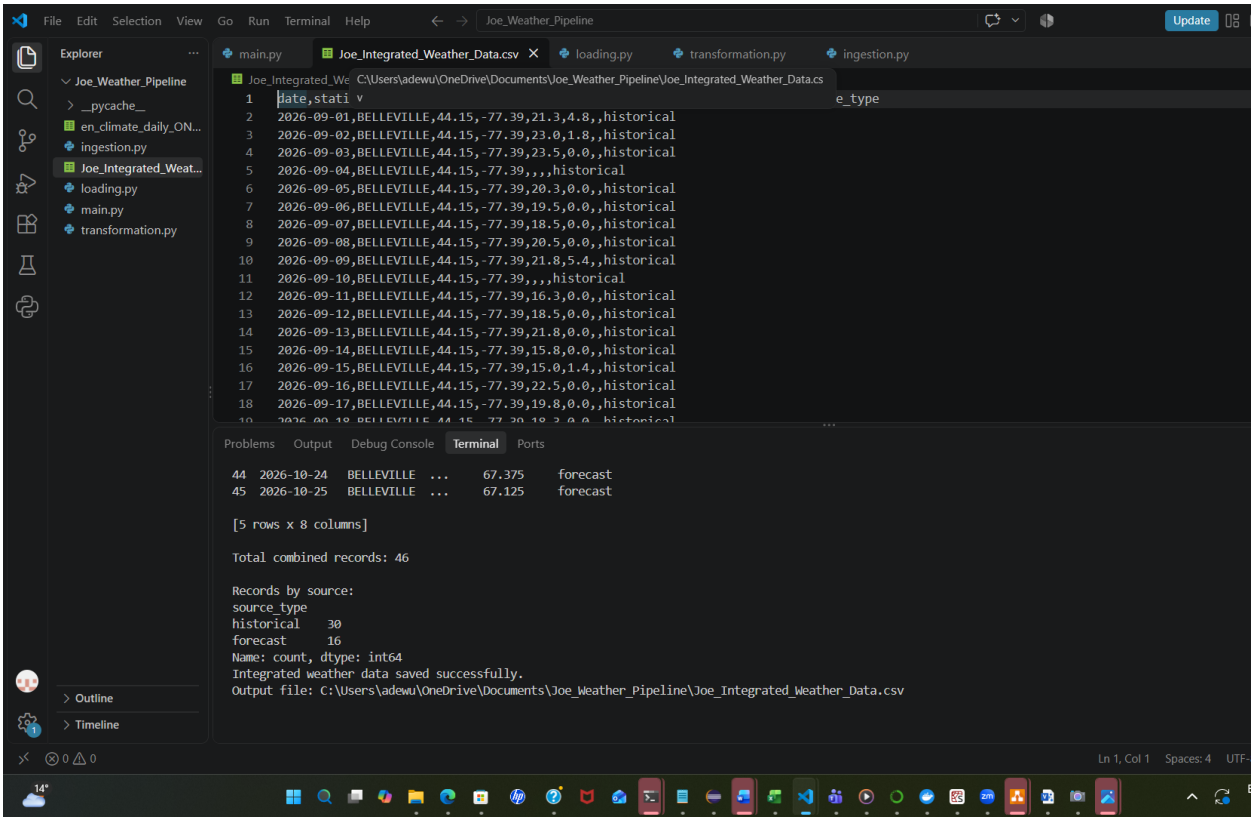


Figure 3.4: Final integrated weather dataset.

3.7. Prototype Summary

The prototype demonstrates the successful implementation of the proposed ETL architecture. By separating data ingestion, transformation, and loading into independent modules, the system provides an organized and maintainable approach to integrating weather information from different sources.

The prototype successfully retrieves forecast information, processes historical observations, standardizes data formats, combines the records, and performs basic data-quality validation.

Although the current implementation uses a manually executed pipeline and CSV storage, the modular architecture provides opportunities for future improvements, including automated scheduling, additional weather-data sources, database integration, and connection to a weather prediction model.

Source Code Repository: The Python prototype and supporting files are available in the following GitHub repository: https://github.com/janet1090/Joe-Weather-Data-Pipeline